

KLASY ABSTRAKCYJNE W JAVIE

Teoria | Składnia | Polimorfizm | Dobre praktyki

Materiały do lekcji // Java SE 17+

6 rozdziałów

Teoria + kod

Pliki Java

Java SE 17+

Skąd się wzięły klasy abstrakcyjne?

Żeby zrozumieć klasy abstrakcyjne, trzeba najpierw zobaczyć **problem który rozwiązują**. Nie pojawiły się w Javie bez powodu — są odpowiedzią na konkretną niedogodność zwykłego dziedziczenia.

Scenariusz — system dla zoo

Wyobraź sobie że projektujesz system dla zoo. Masz klasę `Zwierze` i po niej dziedziczą `Pies`, `Kot`, `Papuga`. Wszystko działa — dopóki ktoś w zespole nie napisze przypadkowo:

Java

```
new Zwierze(); // Co to jest? Jakie dźwięki wydaje? Jak wygląda?
```

Taki obiekt nie powinien istnieć, bo **"zwierzę w ogóle"** to pojęcie abstrakcyjne, nie konkretna istota. Zwykłe dziedziczenie nie daje żadnego narzędzia żeby to zablokować. Klasa abstrakcyjna właśnie to robi — **zabrania tworzenia instancji klasy która ma być tylko szablonem**.

Czym jest klasa abstrakcyjna?

Klasa abstrakcyjna to klasa która **istnieje wyłącznie po to żeby być dziedziczona**. Sama w sobie jest niepełna — celowo. Opisuje ogólny byt, który w rzeczywistości zawsze przybiera konkretną formę.

5 kluczowych właściwości

1. Nie można jej instancjonować

Nie da się napisać `new Zwierze()` jeżeli `Zwierze` jest abstrakcyjna. Kompilator natychmiast zgłosi błąd. To nie jest ograniczenie — to **zabezpieczenie przed błędami projektowymi**.

2. Może zawierać metody abstrakcyjne

Metoda abstrakcyjna to metoda **bez ciała** — sama deklaracja że 'coś takiego istnieje', bez żadnej implementacji. Każda podklasa musi ją nadpisać.

3. Może zawierać zwykłe metody

I tu jest kluczowa różnica — klasa abstrakcyjna może mieć w pełni **zaimplementowane metody** które podklasy dziedziczą bez zmian. Wspólna logika żyje w jednym miejscu.

4. Może mieć konstruktor

Choc nie można jej instancjonować bezpośrednio, konstruktor jest wywoływany przez podklasy przez `super()`. Służy do inicjalizacji wspólnych pól.

5. Może mieć pola

Zwykłe pola z dowolnymi modyfikatorami dostępu — `private`, `protected`, `public`. To kolejna różnica w stosunku do interfejsów.

Cecha	Klasa zwykła	Klasa abstrakcyjna	Interfejs
Instancjonowanie	tak	NIE	NIE
Metody abstrakcyjne	nie	tak (opcjonalne)	tak (domyslnie)
Metody zaimplementowane	tak	tak	tylko default
Konstruktor	tak	tak	NIE
Pola instancji	tak	tak	NIE (tylko stałe)
Dziedziczenie	extends (1)	extends (1)	implements (wiele)

Metoda abstrakcyjna

Metoda abstrakcyjna to **obietnica złożona podklasom** — 'ta operacja istnieje, ale każdy z was zrobi ją po swojemu'. To serce całego mechanizmu klas abstrakcyjnych.

Składnia

```
Java
// Metoda abstrakcyjna – brak ciała, tylko sygnatura
public abstract void wydajDzwiek();
// Metoda zwykła – ma ciało, jest dziedziczona
public void spij() {
    System.out.println(imie + " spi...");
}
```

Trzy reguły metod abstrakcyjnych

- 1. Tylko w klasie abstrakcyjnej.** Zwykła klasa nie może mieć metod abstrakcyjnych — byłyby to sprzecznością, bo zwykłą klasę można instancjonować, a obiekt z 'dziurą' w metodzie nie może działać.
- 2. Każda podklasa musi ją nadpisać.** Jeżeli podklasa tego nie zrobi, sama musi być abstrakcyjna — inaczej kompilator zgłosi błąd.
- 3. Adnotacja @Override jest wymagana.** Informuje kompilator że nadpisujemy metodę. Jeżeli popełnimy literówkę w nazwie — kompilator wykryje błąd. Bez @Override przeszłoby to jako nowa metoda.

Metoda final — odwrotność abstrakcyjnej

Słowo `final` zabrania nadpisywania metody w podklasach — dokładne odwrócenie `abstract`. Używamy gdy logika musi być identyczna w całej hierarchii.

```
Java
// Tej metody ZADNA podklasa nie może nadpisać
public final String toString() {
    return rodzaj() + " [" + imie + ", " + wiek + " lat]";
}
```

Jak klasa abstrakcyjna modeluje rzeczywistość?

Klasa abstrakcyjna modeluje **kategorie bytów które istnieją tylko jako konkretne egzemplarze**. 'Pojazd' sam w sobie nie istnieje — istnieje samochód, motocykl, rower. 'Figura geometryczna' nie istnieje — istnieje koło, kwadrat, trójkąt.

Relacja "jest" jako fundament

Klasa abstrakcyjna zawsze wyraża relację "jest":

- `Pies extends Zwierze` — Pies **jest** Zwierzęciem
- `Koło extends Figura` — Koło **jest** Figurą
- `Samochod extends Pojazd` — Samochód **jest** Pojazdem

Ta relacja ma konkretne konsekwencje. Jeżeli Pies dziedziczy po Zwierze, to obiekt klasy Pies może być używany wszędzie tam gdzie oczekiwany jest typ Zwierze. Kompilator to akceptuje bo pies naprawdę jest zwierzęciem.

Hierarchia wielopoziomowa

Abstrakcyjność się dziedziczy. Można budować wielopoziomowe hierarchie gdzie każdy poziom dodaje trochę konkretności:

Java

```
// Poziom 1 – najbardziej ogólny
abstract class Pojazd {
    protected String marka;
    public abstract void rusz();
}

// Poziom 2 – bardziej szczegółowy, ale wciąż abstrakcyjny
abstract class PojazdSilnikowy extends Pojazd {
    protected int pojemnoscSilnika;
    public abstract String typPaliwa();
    public void uruchomSilnik() {
        System.out.println(marka + ": Vroom!");
    }
}

// Poziom 3 – konkretny, implementuje wszystko
class Samochod extends PojazdSilnikowy {
    @Override public void rusz() { /* implementacja */ }
    @Override public String typPaliwa() { return "benzyna"; }
}
```

Polimorfizm z klasami abstrakcyjnymi

Klasa abstrakcyjna jest naturalnym punktem gdzie **polimorfizm** się zaczyna. Jedna zmienna typu abstrakcyjnego może wskazywać na obiekty różnych konkretnych podklas.

Polimorfizm w akcji

```
Java
// Tablica typu Zwierze[] – może przechowywać Psa, Kota, Papugę
Zwierze[] zoo = {
    new Pies("Rex", 3, "Owczarek"),
    new Kot("Luna", 5, true),
    new Papuga("Polly", 2, "Ciasteczko")
};
// Java sama wybiera właściwą implementację wydajDzwiek()
for (Zwierze z : zoo) {
    z.wydajDzwiek(); // dynamiczne wiązanie!
}
// Wynik:
// Rex: Hau hau!
// Luna: Mrrr... miau
// Polly: Ciasteczko! Skraaak!
```

Dynamiczne wiązanie

Dynamiczne wiązanie to mechanizm przez który Java decyduje którą wersję metody wywołać **w czasie wykonania** (nie kompilacji), na podstawie faktycznego typu obiektu — nie typu zmiennej.

Aspekt	Kompilacja	Wykonanie
Typ zmiennej	Zwierze (sprawdzany przez kompilator)	—
Typ obiektu	—	Pies / Kot / Papuga (faktyczny)
Metoda	wydajDzwiek() z Zwierze (czy istnieje)	wydajDzwiek() z Pies/Kot/Papuga

instanceof i rzutowanie

Czasem mamy zmienną typu nadklasy i chcemy dostać się do metody specyficznej dla podklasy. Używamy `instanceof` do sprawdzenia typu, a następnie wykonujemy rzutowanie:

Java

```
for (Zwierze z : zoo) {  
    if (z instanceof Pies) {  
        // Java 16+: pattern matching  
        Pies p = (Pies) z;  
        p.aportuj(); // metoda dostępna tylko w Pies  
    }  
}
```

Co klasa abstrakcyjna daje projektantowi?

Trzy konkretne wartości

1. Wymuszenie spójności

Każda podklasa **musi** zaimplementować metody abstrakcyjne — bez wyjątków. Nie ma możliwości żeby ktos w zespole zapomniał dodać metodę `wydajDzwiek()` do nowego zwierzęcia. Kompilator na to nie pozwoli.

2. Eliminacja duplikacji

Wspólna logika — pola, konstruktory, zaimplementowane metody — żyje w jednym miejscu. Zmiana w klasie abstrakcyjnej automatycznie **propaguje się do wszystkich podklas**. Nie ma ryzyka że jedna podklasa zostanie pominięta przy aktualizacji.

3. Wyraźny kontrakt hierarchii

Klasa abstrakcyjna mówi każdemu kto czyta kod: 'tutaj zaczyna się rodzina powiązanych klas, oto co mają wspólnego, oto co każda musi zdefiniować osobno'. To **dokumentacja wbudowana w strukturę programu**.

Kiedy NIE używać klasy abstrakcyjnej

- **Gdy klasy nie łączy relacja 'jest'**. Jeżeli jedynym powodem dziedziczenia jest chęć współdzielenia kodu — to zły powód. Lepszym rozwiązaniem jest wtedy kompozycja.
- **Gdy wspólne zachowanie dotyczy klas z różnych rodzin**. Jeżeli chcemy opisać że Kaczka i Samolot potrafią latać — tu wchodzi interfejsy, nie klasy abstrakcyjne.
- **Gdy hierarchia jest płytka i prosta**. Nie każda klasa potrzebuje abstrakcyjnego przodka. Nadmierne używanie prowadzi do sztywnej, trudnej do zmiany struktury.